

MODULE 10

The secure bridge

Named CORS · Vite dev proxy · environment.apiUrl · OPTIONS in DevTools

TMS full stack

```
policy.WithOrigins(origins!)  
  
    .AllowCredentials(); // named CORS  
  
"apiUrl": "/api/v1" // Vite proxy
```

Same TMS one origin in dev, explicit origins in prod

Session outcomes

By the end of this session, you will:

- Configure named CORS policies in .NET 10 from configuration
- Use the Vite dev proxy so /api and /hubs reach .NET without CORS fights
- Implement environment files for URL switching Dev vs Prod
- Debug CORS preflight (OPTIONS) failures in Chrome DevTools
- Replace hardcoded origins with production-ready configuration bindings

The architecture bridge

- Dev: Vite proxies /api (and /hubs) browser sees one origin
- Prod: named CORS policy explicit allowed origins + credentials when cookies are used
- Avoid: AllowAnyOrigin() + credentials browsers reject; wrong habit for production

Mermaid in prose

Dev: Angular :4200 → same-origin /api → Vite forwards to .NET :5000. Prod: Angular host → direct HTTPS to .NET with Access-Control-Allow-Origin from a named policy.

.NET 10 CORS named policy from config

Program.cs (trimmed)

```
// Program.cs bind to appsettings.json

var origins = builder.Configuration.GetSection("CORS:AllowedOrigins").Get<string[]>();

builder.Services.AddCors(opt => opt.AddPolicy("TmsClient", policy =>
    policy.WithOrigins(origins!)
        .AllowAnyMethod()
        .AllowAnyHeader()
        .AllowCredentials())); // cookie auth

app.UseCors("TmsClient"); // after Routing, before Auth
```

- Origins from configuration staging/production URLs change without recompile

The AllowAnyOrigin trap

Anti-pattern

```
// THROWS InvalidOperationException AT STARTUP

policy.AllowAnyOrigin()
    .AllowCredentials(); // forbidden combination
```

- AllowAnyOrigin = any site on earth; AllowCredentials = send cookies together = stolen session risk
- .NET refuses to start security check, not a bug

Exact origin

```
// The fix

policy.WithOrigins("http://localhost:4200")
    .AllowCredentials();
```

Vite functional proxy

proxy.conf.json

```
// proxy.conf.json  local dev bridge

{
  "/api": {
    "target": "http://localhost:5000",

    "secure": false,

    "changeOrigin": true,

    "logLevel": "debug"
  },
  "/hubs": {
    "target": "http://localhost:5000",

    "secure": false,

    "ws": true
  }
}
```

- "ws": true on /hubs otherwise SignalR WebSockets fail silently in dev
- Angular calls /api/courses → Vite forwards to .NET:5000 browser sees one domain

Environment switching

Angular environments

```
// environment.development.ts (Development)
export const environment = {
  production: false,

  apiUrl: "/api/v1",
};

// environment.ts (Production)
export const environment = {
  production: true,

  apiUrl: "/api/v1",
};
```

- Services read environment.apiUrl file replacements swap dev proxy path vs full prod base
- No hardcoded localhost:5000 strings in feature services

Common mistakes Session 1

Mistake	Symptom	Fix
AllowAnyOrigin() + AllowCredentials()	InvalidOperationException at startup	Use WithOrigins("exact-url")
CORS after UseAuthorization()	Preflight OPTIONS returns 401	Move UseCors() before UseAuthentication()
Missing "ws": true on /hubs proxy	SignalR WebSocket silently fails in dev	Add "ws": true to the proxy entry
Hardcoded localhost:5000 in Angular	Breaks outside local dev	Use environment.apiUrl
Proxy edit without restart	Proxy changes not applied	Stop and restart ng serve

Session 1 summary

- CORS is a browser security guard not a mysterious server error
- Vite proxy: same-origin dev traffic so CORS does not slow the inner loop
- Named CORS policies in .NET protect explicit allowed origins in production
- Environment files keep the API base URL out of feature code
- Middleware order on the API: the classic "works in Postman, fails in the browser" gap

Interview angle

Why SPA hits CORS but Postman does not how you pair dev proxy with named production policies when credentials matter.

Next: Session 2 HttpOnly cookies and XSRF

Cookie auth, antiforgery, X-XSRF-TOKEN TMS mutations stay safe across sites.